

Informe de Avance – Trabajo Práctico Final

Algoritmos y Estructuras de Datos - Comisión 6

Grupo: Alan leslie, Axel Sanchez, Tomás Guerrero

Introducción

En esta primera etapa del Trabajo Práctico Final estuvimos trabajando principalmente en entender bien el problema planteado y en organizar cómo íbamos a desarrollar la aplicación. El objetivo del trabajo es crear un sistema que permita analizar datos obtenidos desde archivos CSV para encontrar cuáles son los elementos que más veces aparecen dentro de esos conjuntos de datos.

La aplicación fue desarrollada en C# y permite al usuario seleccionar un archivo CSV, cargarlo en memoria y realizar búsquedas utilizando dos estrategias distintas: una basada en una estructura Heap Binaria y otra basada en un algoritmo de ordenamiento. Además, la aplicación ofrece consultas adicionales sobre la estructura interna de la Heap construida.

Descripción del trabajo

El trabajo consiste en completar una clase llamada Estrategia que contiene cinco métodos principales:

Método	Descripción
BuscarConHeap	Encuentra los N elementos más frecuentes usando una Max-Heap.
BuscarConOrden	Misma funcionalidad pero usando un algoritmo de ordenamiento.
Consulta1	Compara los tiempos de ejecución de ambos métodos.
Consulta2	Devuelve el camino a la hoja más

	izquierda de la Heap.
Consulta3	Muestra los datos de la Heap organizados por niveles.

BuscarConHeap:

Este método fue implementado en su totalidad. Consta de tres pasos: primero, cuenta las ocurrencias de cada elemento usando un diccionario; segundo, construye una Max-Heap insertando cada elemento con su frecuencia; tercero, extrae los N elementos con mayor cantidad de ocurrencias y los devuelve en la lista collected.

Para el correcto funcionamiento de la Heap se implementaron dos métodos auxiliares privados: FiltrarArriba, que restaura la propiedad de la Heap luego de una inserción, y FiltrarAbajo, que la restaura luego de una extracción del máximo.

BuscarConOtro:

Este método fue implementado utilizando un algoritmo de ordenamiento basado en Selection Sort. Al igual que BuscarConHeap, primero cuenta las ocurrencias, luego ordena la lista de mayor a menor por frecuencia y finalmente toma los primeros N elementos para devolverlos en collected.

Consulta 1:

Este método mide y compara los tiempos de ejecución de BuscarConHeap y BuscarConOtro al buscar los 5 elementos con mayor cantidad de ocurrencias. Se utilizó la clase Stopwatch de C# para medir el tiempo en milisegundos de cada método por separado, retornando ambos resultados en un único string.

Problemas Encontrados y Soluciones

Nombre del método en el Backend:

La universidad proveyó un archivo Backend.cs que llama al método BuscarConOtro, mientras que inicialmente el método había sido llamado BuscarConOrden en el enunciado del proyecto. Esto generó un error de compilación que se resolvió renombrando el método para que coincidiera con lo esperado por el Backend.

Conclusión final

En esta primera etapa se logró implementar y probar correctamente tres de los cinco métodos solicitados: BuscarConHeap, BuscarConOtro y Consulta1. Los tres producen resultados correctos y fueron verificados con datos de prueba y con el dataset provisto por la cátedra.

Los conceptos trabajados incluyen estructuras de datos como Heap Binaria Max, diccionarios, listas dinámicas, algoritmos de ordenamiento vistos en clase y medición de tiempos de ejecución. Quedan pendientes para la segunda entrega la implementación de Consulta2 y Consulta3.